

Actividad 1 · Pipeline de scikit-learn

Link al repo: <https://github.com/DiegoLinares11/Actividad1-MLOPS>

Integrantes:

Diego Linares · Andy Fuentes · Christian Echeverria · Diederich Solis

Dataset: `champions_league_matches.csv` — el mismo que analizamos en el Ejercicio 1.

En el Ejercicio 1 hicimos el análisis exploratorio del dataset y propusimos un modelo: *predecir el resultado del partido (clasificación multiclase)*. Esta actividad es la continuación: construimos el pipeline de preparación de datos para ese modelo y mostramos su diagrama.

Todo el desarrollo se enmarca en las dos primeras etapas de **CRISP-DM**.

1. Business Understanding (Comprensión del negocio)

Queremos **anticipar el resultado de un partido de Champions League** a partir de las estadísticas de juego (posesión, tiros a puerta, atajadas).

- **Objetivo:** clasificar cada partido como `Home Win`, `Away Win` o `Draw`.
- **Para qué sirve:** análisis deportivo, previas de partidos, apoyo a decisiones tácticas.
- **Criterio de éxito:** superar la clase mayoritaria (el local gana ~49% de los partidos) y que el pipeline sea reproducible por todo el equipo en cualquier computadora.

2. Data Understanding (Comprensión de los datos)

Del análisis del Ejercicio 1 ya sabemos lo siguiente, y es justo lo que el pipeline tiene que resolver:

Hallazgo del Ejercicio 1	Cómo lo atiende el pipeline
El CSV trae 7 filas totalmente vacías que separan jornadas	Se eliminan en el filtrado (quedan 144 partidos)
La posesión viene como texto <code>'63%'</code>	Transformador <code>PorcentajeATexto</code>
Tiros y atajadas vienen como texto <code>'3 of 10'</code>	Transformador <code>RatioATexto</code> (genera <i>hechos e intentos</i>)
<code>home_saves_pct</code> tiene algunos nulos (casos de 0 de 0)	Imputación por mediana
Los equipos son variables categóricas	One-Hot Encoding

Ojo con la fuga de información (*data leakage*): las columnas `score` y `winner` **revelan el resultado**. Si las dejáramos como variables de entrada, el modelo tendría 100% de acierto pero sería inútil en un partido que aún no termina. Por eso se descartan en el filtrado.

Variable objetivo: `result` (Home Win: 71, Away Win: 48, Draw: 25).

```
In [ ]: from sklearn import set_config

# Hace que los estimadores se muestren como DIAGRAMA (no como texto)
set_config(display="diagram")

from act1_pipeline import (
    extraer_datos, filtrar_datos, separar_datos, construir_pipeline,
    NUMERIC_FEATURES, PERCENT_FEATURES, RATIO_FEATURES, CATEGORICAL_FEATURES,
    LEAKAGE_COLUMNS, TARGET,
)
```

Etapas 1 — Extracción

Se carga el CSV de partidos que viene empaquetado dentro de la librería.

```
In [ ]: df = extraer_datos()
print("Forma del CSV crudo:", df.shape)
df.head()
```

Forma del CSV crudo: (151, 18)

```
Out [ ]:
```

	date	home_team	away_team	score	venue	referee	home_possession	awa
0	2025-09-16	PSV	Union SG	1-3	Philips Stadion	Anthony Taylor	63%	
1	2025-09-16	Athletic Club	Arsenal	0-2	San Mamés	Donatas Rumšas	38%	
2	2025-09-16	Tottenham Hotspur	Villarreal	1-0	Tottenham Hotspur Stadium	Rade Obrenović	58%	
3	2025-09-16	Benfica	Qarabağ	2-3	Estádio do Sport Lisboa e Benfica	Erik Lambrechts	53%	
4	2025-09-16	Juventus	Dortmund	4-4	Allianz Stadium	François Letexier	52%	

```
In [ ]: # Se confirman los dos problemas detectados en el Ejercicio 1
print("Filas totalmente vacías:", int(df.isna().all(axis=1).sum()))
print("\nColumnas numéricas guardadas como texto:")
df[["home_possession", "home_shots_on_target", "home_saves"]].head(3)
```

Filas totalmente vacías: 7

Columnas numéricas guardadas como texto:

```
Out [ ]:
```

	home_possession	home_shots_on_target	home_saves
0	63%	3 of 10	4 of 8
1	38%	2 of 11	4 of 6
2	58%	1 of 9	0 of 0

Etapla 2 — Filtrado

Se eliminan las filas vacías, las columnas que revelan el resultado (`score` , `winner`) y las que no aportan al modelo (`date` , `venue` , `referee`).

```
In [ ]: df_filtrado = filtrar_datos(df)

print("Antes: ", df.shape)
print("Después:", df_filtrado.shape)
print("\nColumnas descartadas por fuga de información:", LEAKAGE_COLUMNS)
print("\nDistribución de la variable objetivo:")
print(df_filtrado[TARGET].value_counts())
```

Antes: (151, 18)

Después: (144, 13)

Columnas descartadas por fuga de información: ['score', 'winner']

Distribución de la variable objetivo:

result

Home Win 71

Away Win 48

Draw 25

Name: count, dtype: int64

Etap 4 — Separación del dataset

Se divide en `X / y` y en entrenamiento y prueba, **estratificando** por `result` para que las tres clases queden representadas en ambos conjuntos (importante porque `Draw` es la clase minoritaria).

```
In [ ]: X_train, X_test, y_train, y_test = separar_datos(df_filtrado)
print("train:", X_train.shape, " test:", X_test.shape)
print("\nProporción de clases en train:")
print(y_train.value_counts(normalize=True).round(3))
```

train: (115, 12) test: (29, 12)

Proporción de clases en train:

result

Home Win 0.496

Away Win 0.330

Draw 0.174

Name: proportion, dtype: float64

Etap 3 — Manejo de los diferentes tipos de variables

Este dataset tiene **cuatro tipos** de variables y cada uno necesita un tratamiento distinto:

Tipo	Columnas	Tratamiento
Numéricas ya limpias	<code>*_pct</code>	imputar (mediana) → estandarizar
Porcentaje en texto	<code>home_possession</code> , <code>away_possession</code>	'63%' → 63.0 → imputar → estandarizar
Razón en texto	<code>*_shots_on_target</code> , <code>*_saves</code>	'3 of 10' → 3.0 y 10.0 → imputar → estandarizar
Categorías	<code>home_team</code> , <code>away_team</code>	imputar (moda) → One-Hot Encoding

```
In [ ]: print("Numéricas: ", NUMERIC_FEATURES)
print("Porcentajes:", PERCENT_FEATURES)
print("Razones:     ", RATIO_FEATURES)
print("Categorías:", CATEGORICAL_FEATURES)
```

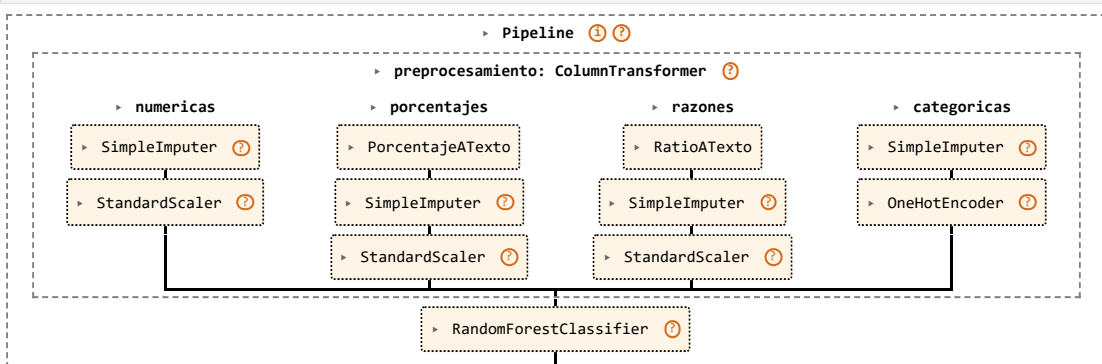
```
Numéricas:  ['home_shots_on_target_pct', 'away_shots_on_target_pct', 'home_saves_pct', 'away_saves_pct']
Porcentajes: ['home_possession', 'away_possession']
Razones:     ['home_shots_on_target', 'away_shots_on_target', 'home_saves', 'away_saves']
Categorías:  ['home_team', 'away_team']
```

Diagrama del pipeline

Al ejecutar la siguiente celda, scikit-learn genera el **diagrama interactivo** del pipeline. Se puede hacer clic en cada bloque para desplegar sus parámetros.

```
In [ ]: pipe = construir_pipeline()
```

```
Out[ ]:
```



Entrenamiento y evaluación

Se entrena el pipeline completo (preprocesamiento + modelo) y se evalúa en el conjunto de prueba.

```
In [ ]: from sklearn.metrics import accuracy_score, classification_report

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)

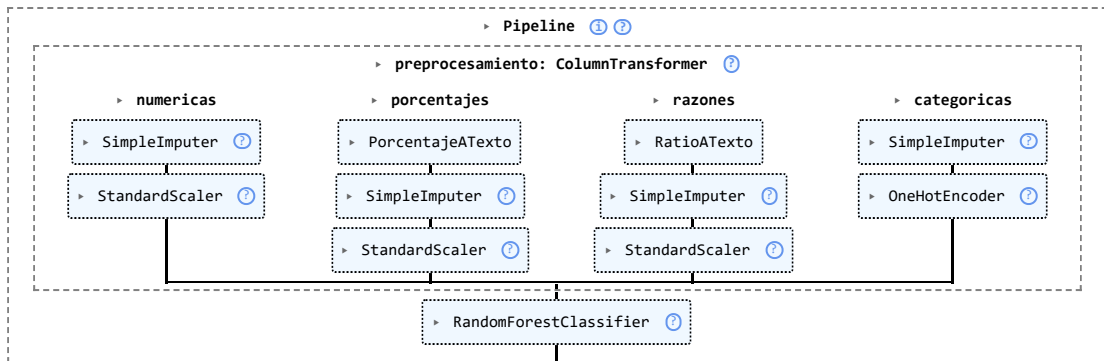
print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
print()
print(classification_report(y_test, y_pred, digits=3))
```

Accuracy: 0.724

	precision	recall	f1-score	support
Away Win	0.615	0.800	0.696	10
Draw	0.500	0.200	0.286	5
Home Win	0.857	0.857	0.857	14
accuracy			0.724	29
macro avg	0.658	0.619	0.613	29
weighted avg	0.712	0.724	0.703	29

```
In [ ]: # El pipeline ya entrenado también se muestra como diagrama (marcado como "Fitted pipe
```

```
Out[ ]:
```



Captura de evidencia

```
act1-demo

=====
ACTIVIDAD 1 - Pipeline de scikit-learn (UEFA Champions League)
=====
Paquete act1_pipeline v0.1.0
Equipo (hostname): WH-Chris
Sistema operativo: Linux 7.0.3-1-cachyos
Python: 3.14.4
Fecha/hora: 2026-07-28 18:57:30

[1] Extracción      -> 151 filas, 18 columnas
[2] Filtrado        -> 144 filas, 13 columnas
[4] Separación      -> train=115 test=29

=====
RESULTADO - Accuracy en prueba = 0.724
=====
```

	precision	recall	f1-score	support
Away Win	0.615	0.800	0.696	10
Draw	0.500	0.200	0.286	5
Home Win	0.857	0.857	0.857	14
accuracy			0.724	29
macro avg	0.658	0.619	0.613	29
weighted avg	0.712	0.724	0.703	29

```
Diagrama del pipeline guardado en: /home/chris/Documents/U/MLOPS/Actividad1-MLOPS/pipeline_diagram.html

[OK] Pipeline ejecutado correctamente en este equipo.
```

Mi compu, diferentes SO y todo y da mismo resultado:

```
PS C:\Users\dlinares\Documents\UVG\Ultimo Semestre\Machine Learning Engineering\actividad 1> python -m act1_pipeline
.cli

=====
ACTIVIDAD 1 - Pipeline de scikit-learn (UEFA Champions League)
=====
Paquete act1_pipeline v0.1.0
Equipo (hostname): Dlinares
Sistema operativo: Windows 11
Python: 3.14.2
Fecha/hora: 2026-07-31 22:06:13

[1] Extracción      -> 151 filas, 18 columnas
[2] Filtrado        -> 144 filas, 13 columnas
[4] Separación      -> train=115 test=29

=====
RESULTADO - Accuracy en prueba = 0.724
=====
              precision    recall  f1-score   support

   Away Win         0.615       0.800       0.696        10
     Draw          0.500       0.200       0.286         5
   Home Win         0.857       0.857       0.857        14

 accuracy                   0.724         29
  macro avg         0.658       0.619       0.613         29
 weighted avg         0.712       0.724       0.703         29

Diagrama del pipeline guardado en: C:\Users\dlinares\Documents\UVG\Ultimo Semestre\Machine Learning Engineering\acti
vidad 1\pipeline_diagram.html

[OK] Pipeline ejecutado correctamente en este equipo.
PS C:\Users\dlinares\Documents\UVG\Ultimo Semestre\Machine Learning Engineering\actividad 1>
```